



Full Stack Development Week 05

Topics covered:

- Router module
- Modify localStorage data in web application with functional components and hooks
- useEffect dependencies

Activity 1: Modify localStorage data in a web application - using functional components & hooks

Consider the following page:

ID	Name	Type
1	Daisy	Dog
2	Saphira	Cat

Pet

ID	Name	Type
1	Daisy	Dog
Update Pet		

The pet data will be stored in localStorage. The form helps the user to update pet data. The updated data will be stored back to localStorage and display table refreshed accordingly.

Activity 2: Routing in Next using the useRouter hook

In this activity, we'll practice using the Next.js router with the useRouter hook.

1. Create a new page component called `about.tsx` in your pages directory
2. Add a simple heading and paragraph about your website
3. In your home page (`index.tsx`), add a button that says "About Us"
4. Import and use the `useRouter` hook from 'next/router'
5. Add an onClick handler to the button that uses `router.push()` to navigate to the about page
6. Test that clicking the button successfully navigates between pages

Next see if you can read the current URL of the page using the `useRouter` hook and display this in the header.

Activity 3: `useEffects` dependency array

From lection 4, we know that the documentation of `useEffect` tells us:

`useEffect` accepts two arguments.

`useEffect(<function>, <dependency>)`

This argument `dependency`:

It's optional. If you don't specify it, the effect runs after each render.

If it's empty (`[]`), the effect runs once, after the initial render.

It must — or should — contain the list of values used in the effect. The effect runs after any of these values changes (and after the initial render).

Discuss the above with the help of a code example with your tutor.

Activity 4: `useContext`

In this activity, we'll explore React's Context API and the `useContext` hook by building a simple shared dollar value tracker.

1. Create a dollar value context:

- Create a new file called `DollarContext.tsx`
- Define a context that tracks a single dollar amount
- Create a `DollarProvider` component with methods to:
 - Add dollar amounts to the total
 - Reset the dollar value

2. Build components:

- Create an `AddDollars` component with an input and button
- Create a `DisplayDollars` component to show current amount
- Both components will use the context to read/update the dollar value

3. Connect everything:

- Wrap your app with the `DollarProvider`
- Add both components to your page
- Test that updates in one component are reflected in the other